

第 6 课 · 答辩与模拟面试

把做过的工程讲成一个能拿 offer 的故事

第 6 课 · 120 分钟（演示 40 + 简历 30 + 模拟面试 40 + 收尾 10）

一、这节课的目标

三件事：第一，你能在五分钟内讲清楚这个项目；第二，你知道这段经历在简历上该怎么写、面试里会被追问什么；第三，做一轮完整的模拟面试。

这节课是演练，不是听讲。每个人都要讲一遍。

二、口语化讲稿

这节课为什么重要

前面五节课你做完了项目。但用人单位看不到你做过的过程，他只能看到简历上那两行字，和面试里你说的那五分钟。所以「会做」和「会讲」之间还有一段距离，这节课就是补这一段。

我先说个残酷的事实：**大部分技术面试挂掉，不是因为不会，是因为没讲清楚**。
面试官一天面八个人，每个人都在说「我做了个项目，用了 RAG 和 agent」。你要做的是让他记住你——而让人记住的办法，是讲一个**反直觉的发现**。

这节课我们练的，就是把你的项目变成一个「面试官会追问」的故事。下面我先把结构给你，然后我们一条条拆。

三、五分钟讲清楚这个项目

结构：背景 → 问题 → 方案 → 数据 → 取舍

我给你一个固定的五分钟结构。不要即兴，就按这五步走。面试官只有五分钟耐心，结构清楚比你讲得多重要得多。

步骤	讲什么	时长
背景	训练/评测平台的运维痛点：任务挂了要查很久	3秒
问题	要自动化的是「查证据 → 判断根因」这套动作	3秒
方案	只读工具 + 混合检索 + 有界循环 + 上游闸门	9秒
数据	评测准确率、检索召回、并发下的拒绝率	9秒
取舍	为什么这么设计、放弃了什么	6秒

第三步和第五步是分水岭。****面试官问不倒你的地方，就是你讲取舍的地方****。只讲「用了什么技术」的人很多，能讲「为什么不用另一个方案」的人很少。

开场白（直接可以背）

「我做过一个 AIOps Agent，用来诊断训练和评测平台上的任务失败。背景是这类故障的排查很耗人——要翻日志、看指标、对平台知识，一次可能二十分钟。

我的方案是让 agent 自己去查：给它一个 job id，它调用只读工具读记录、日志、指标，用混合检索找到对应的 runbook，然后给出根因、证据和修复建议，证据不足时会拒答而不是硬猜。

工程上我主要解决了三个问题：一是上游模型服务的并发保护，二是检索质量怎么量化，三是怎么评测一个没有唯一标准答案的 agent。总共大约一万行代码，159 个测试。」

注意最后那句「159 个测试」。有测试的项目和没有测试的项目，在面试官眼里是两个物种。

四、简历怎么写

错误写法 vs 正确写法

错误（占 90%）	正确
使用 LLM 技术构建智能问答系统	实现了面向训练平台故障诊断的 Agent，端到端评测准确率 98.3%
熟悉 RAG 检索增强生成	在 286 条真实故障日志上量化检索质量：纯词法 top-1 82.5%，发现了等权融合会导致 25 个点的下降，据此确定融合权重
具备高并发系统经验	设计上游准入闸门，使模型服务在 3 倍超载下仍保持 120/120 请求成功，超出的负载明确拒绝而非拖垮上游

错误（占 90%）	正确
了解大模型评测	设计了包含「拒答」类别的分层评测集，同时报告准确率与校准指标，避免「全部拒答」刷满准确率的作弊路径

看到区别了吗？右边每一条都有一个**具体数字**，或者一个**具体的反直觉发现**。
左边的每一条，一千份简历里都一样。

最值钱的那条是第三条里的「25 个点的下降」。因为它证明你**真的动手测过**，而不是照着教程抄了一遍。反直觉的发现是面试官唯一会追问的东西。

三条可以量化的事实

这三个数字是本项目最硬的产出，背下来。

- 1. **286 条真实故障日志上的检索对比**：纯词法 82.5%、纯向量 29%、等权融合 57%。结论是融合权重必须偏向强的那一侧。
- 2. **7 条拒答守卫的纵深防御**：全部禁用后，未知类别准确率从 100% 掉到 0%、整体从 100% 掉到 81.7%。说明拒答是设计出来的，不是兜底。
- 3. **3 倍超载下的闸门**：burst 场景 32 个并发客户端打 4 槽队列，系统返回明确拒绝而非上游崩溃，`in\_flight` 始终不超上限。

五、模拟面试

本轮就是本课的小面试环节。每人轮流被追问，其余人记录问题。

第一轮：技术面（20 分钟）

**Q1：白板写一个 agent loop。 **

评分点只有三个：有步数上限、工具结果追加回消息列表（含助手的工具调用）、工具异常被捕获并变成消息。这三个漏一个就不及格。

Q2：你的 RAG 数据从哪来？怎么保证质量？

三层答：来源（从故障剧本派生，天然一致）、质量（字段完整性由测试强制）、评测（286 条真实日志量召回）。详见第 3 课。

Q3：模型服务挂了怎么办？

先分层（客户端/网关/推理服务），再判断是不是重试风暴，然后看上游容量和降级状态，最后才怀疑模型本身。详见第 4 课。

Q4：怎么证明你改完变好了？

固定 golden set、分层采样、对比混淆矩阵而不只看总准确率、把用例清单和配置一起存下来保证可复跑。详见第 5 课。

Q5：这个项目最大的技术难点是什么？

这题没有标准答案，但有一个好答案：「难点不在写 agent，而在识别出**哪些问题不是 agent 问题**」。比如我一开始以为瓶颈是提示词，后来发现是检索把噪声当成了证据；以为要优化模型调用，实际上要优化的是并发闸门。这个项目教给我的是先做度量、再决定改哪里。」

「先度量、再改」这六个字，是整门课想传达的核心工作方法。

第二轮：行为面（15 分钟）

用 **STAR**

结构答。Situation（情境）、Task（任务）、Action（行动）、Result（结果）。

Q：讲一次你从零做一个东西的经历。

要示范素	
S	平台上的故障排查全靠人工，一次要二十分钟

要素示范	
T	我想验证「这件事能不能自动化」
A	我先把故障模式整理成一份剧本（29 个场景），因为这决定了整个系统的边界；然后做最小可用版本先跑通闭环；再逐步加上并发保护和评测
R	端到端准确率 98.3%，一份可复跑的评测，159 个测试

注意 A 那一步。我说的是「我先整理故障模式」——这体现了工程判断：****先定边界，再写代码****。新手会直接开始写 agent loop，写到一半发现数据没准备好。

****Q：讲一次你和别人的技术分歧。 ****

- 要具体，不要讲「大家都很好沟通」这种废话
- 结构：分歧是什么 → 双方的理由 → 怎么决定 → 结果如何
- 最好的答案是「我们用数据决定的」——正好接上这个项目
- 退一步说：如果你没有真实经历，就用这个项目里的技术取舍来编一个（比如融合权重之争），但一定要能自圆其说

第三轮：反问环节

面试最后一定会问「你有什么想问的」。这题****不是客套****，是在考察你。

不要问	可以问
你们用什么技术栈？	这个岗位做的系统，目前最大的瓶颈是什么？

不要问	可以问
工作时间怎么样？	团队怎么衡量一次改动的效果？有评测集吗？
我能学到什么？	如果我来了，前三个月最希望我解决什么问题？

右边的问法传递的信息是：「我是来解决问题的，不是来上课的」。尤其第一个，「最大的瓶颈是什么」——这个问题会让面试官觉得你已经站在工程师的位置上思考了。

六、答辩：演示你的项目

五分钟演示脚本

1. 跑`make demo`，展示一次完整诊断，指出 agent 调了哪些工具（1 分钟）
2. 打开 Web UI，按故障注入按钮，展示降级和不降级的对比（1 分钟）
3. 跑`ailab-ops compare`，展示检索质量是被量化的（1 分钟）
4. 跑`make eval`，展示评测报告，重点讲校准指标（1 分钟）
5. 打开`faults.yaml`，讲「一份剧本驱动全局」的设计（1 分钟）

第五步很多人会跳过，但它最能体现架构能力：一个文件加一个场景，生成器、知识库、推理、评测、测试全部自动跟上。这叫做**单一事实来源**。

可能被追问的问题，提前准备好

- 「你这个准确率 100%，是不是有问题？」→ 答：数据集和评分器同源，这是闭环评测；能说明架构正确性，不能外推到真实数据。这正是我接真实模型后端的原因。
- 「为什么不直接用现成的 agent 框架？」→ 答：框架解决工具调用和状态管理，不解决并发、限流、成本、评测。后面这些才是这个项目的主体。
- 「这个能上线吗？」→ 答：能，但需要三件事——接真实模型后端、把会话存储换成共享存储、加鉴权。这三处代码里都标了位置。

- 「数据是假的，有什么意义？」→ 答：故障模式是真的，工程问题是真的一样存在。而且假数据让评测可复现、可作弊检测——真数据反而做不到这一点。

七、收尾：这个项目教了什么

最后一句话总结整门课。Agent 系统的难点从来不是那个循环。

难的是：知道上游会在什么条件下崩、知道检索什么时候在骗你、知道怎么用一个数字证明自己变好了。这三件事有个共同点——**都要求你先做度量，再动手改**。这是这门课真正想教的东西，比任何具体技术都活得久。

八、课后

- 把五分钟讲稿写下来，对着计时器练三遍，录下来自己看
- 把简历里这个项目改写成上面「正确写法」的格式
- 找同学互相模拟面试一轮，交换追问
- 如果你要接真实模型后端练手：``make serve` + `ailab-ops llm-stub``，然后换掉 ``AILAB_LLM_BACKEND=openai``